

1. Grid Data with Pagination

Create a stored procedure to retrieve grid data using multiple input parameters for filtering.

The result should support pagination (e.g., page number and page size).

```
USE [AdManager_Manish]
GO
/***** Object:  StoredProcedure [Inv].[SpScreenSel]      Script Date: 3/15/2026 5:49:05
PM *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
-- =====
-- Author:      Manish Neupane
-- Create date: 13.03.2026
-- Description: Paginated Screen grid with optional filters.
-- Input   : { Offset, PageSize, Filter: { Status, ScreenType, PlacementType, TenantId
} }
-- Output  : { Data: [...], TotalCount: N }
/*
DECLARE @Json NVARCHAR(MAX) = N'{
    "Offset": 0,
    "PageSize": 10,
    "Filter": {
        "ScreenType": "outdoor",
        "Status": "active",
        "PlacementType": null,
        "TenantId": null
    }
}'
EXEC Inv.SpScreenSel @Json;
*/
-- =====
ALTER PROCEDURE [Inv].[SpScreenSel]
(
    @Json NVARCHAR(MAX)
)
AS
BEGIN
    BEGIN TRY
        SET NOCOUNT ON;

        DECLARE
            @Offset      INT          = ISNULL(TRY_CAST(JSON_VALUE(@Json,
'$$.Offset') AS INT), 0),
            @PageSize    INT          = ISNULL(TRY_CAST(JSON_VALUE(@Json,
'$$.PageSize') AS INT), 10),
            @TenantId    INT          = TRY_CAST(JSON_VALUE(@Json,
'$$.Filter.TenantId') AS INT),
            @ScreenType  NVARCHAR(20) = JSON_VALUE(@Json, '$$.Filter.ScreenType'),
            @PlacementType NVARCHAR(20) = JSON_VALUE(@Json, '$$.Filter.PlacementType'),
            @Status      NVARCHAR(20) = JSON_VALUE(@Json, '$$.Filter.Status'),
            @DataJson    NVARCHAR(MAX),
            @TotalCount  INT;

        -- Temp table to stage grid results
        CREATE TABLE #ScreenGrid (
            Id          INT,
            TenantId    INT,
            TenantName  NVARCHAR(100),
```

```

        ScreenName      NVARCHAR(100),
        ScreenType       NVARCHAR(20),
        PlacementType    NVARCHAR(20),
        BasePrice        DECIMAL(10,2),
        Status           NVARCHAR(20),
        MacAddress        NVARCHAR(17),
        Resolution        NVARCHAR(20),
        Orientation       NVARCHAR(10),
        ScreenSizeInch    DECIMAL(5,2),
        Manufacturer      NVARCHAR(100),
        CreatedAt         DATETIME,
        CreatedBy         INT
    );

-- Populate grid with filtered results
INSERT INTO #ScreenGrid (
    Id,
    TenantId,
    TenantName,
    ScreenName,
    ScreenType,
    PlacementType,
    BasePrice,
    [Status],
    MacAddress,
    Resolution,
    Orientation,
    ScreenSizeInch,
    Manufacturer,
    CreatedAt,
    CreatedBy
)

SELECT  s.Id,
        s.TenantId,
        t.TenantName,
        s.ScreenName,
        s.ScreenType,
        s.PlacementType,
        s.BasePrice,
        s.Status,
        si.MacAddress,
        si.Resolution,
        si.Orientation,
        si.ScreenSizeInch,
        si.Manufacturer,
        s.CreatedAt,
        s.CreatedBy

FROM Inv.Screen      s
JOIN Inv.ScreenInfo si ON s.Id      = si.ScreenId
JOIN Core.Tenant     t  ON s.TenantId = t.Id
WHERE s.IsDeleted = 0
      AND (@TenantId IS NULL OR s.TenantId      = @TenantId)
      AND (@ScreenType IS NULL OR s.ScreenType    = @ScreenType)
      AND (@PlacementType IS NULL OR s.PlacementType = @PlacementType)
      AND (@Status IS NULL OR s.Status            = @Status);

-- Build paginated JSON output
SELECT @TotalCount = COUNT(*) FROM #ScreenGrid;

SET @DataJson = (
    SELECT *
    FROM #ScreenGrid

```

```

        ORDER BY CreatedAt DESC
        OFFSET @Offset ROWS
        FETCH NEXT @PageSize ROWS ONLY
        FOR JSON PATH
    );

    SELECT '{"Data":"' + ISNULL(@DataJson, '[]') + '","TotalCount":"' +
    CAST(@TotalCount AS NVARCHAR(10)) + '"}';

    DROP TABLE IF EXISTS #ScreenGrid;

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH
END;

```

2. Bulk Insert with Related Table

Create a stored procedure to bulk insert records into a main table, where the main table is linked to a secondary (related) table.

Ensure related records are inserted correctly, and handle duplicate entries.

```

USE [AdManager_Manish]
GO
/***** Object:  StoredProcedure [Inv].[SpScreenIns]      Script Date: 3/15/2026 5:41:30
PM *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
-- =====
-- Author:      Manish Neupane
-- Create date: 14.03.2026
-- Description: Insert a new Screen and its ScreenInfo. Rejects duplicate MacAddress.
-- Input  : { TenantId, ScreenName, ScreenType, PlacementType, BasePrice, Status,
CreatedBy,
--          ScreenInfo: { MacAddress, Resolution, Orientation, ScreenSizeInch,
Manufacturer } }
-- Output : Inserted Screen record as JSON array
/*
DECLARE @Json NVARCHAR(MAX) = '{
    "TenantId":      1,
    "ScreenName":    "Main Hall Display",
    "ScreenType":    "indoor",
    "PlacementType": "wall",
    "BasePrice":     750.00,
    "Status":        "active",
    "CreatedBy":     1,
    "ScreenInfo": {
        "MacAddress": "AA:BB:CC:DD:EE:01",
        "Resolution": "1920x1080",
        "Orientation": "landscape",
        "ScreenSizeInch": 55.0,
        "Manufacturer": "Samsung"
    }
}'
EXEC Inv.SpScreenIns @Json OUTPUT;
SELECT @Json AS JsonOutput;

```

```

*/
-- =====
ALTER PROCEDURE [Inv].[SpScreenIns]
(
    @Json NVARCHAR(MAX) OUTPUT
)
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;
        SET NOCOUNT ON;

        -- Temp table to capture inserted Screen Id
        CREATE TABLE #Inserted (Id INT);

        -- Temp table to stage parsed JSON data
        CREATE TABLE #Screen (
            TenantId INT NOT NULL,
            ScreenName NVARCHAR(100) NOT NULL,
            ScreenType NVARCHAR(20) NOT NULL,
            PlacementType NVARCHAR(20) NOT NULL,
            BasePrice DECIMAL(10,2) NOT NULL,
            Status NVARCHAR(20) NOT NULL,
            CreatedBy INT NOT NULL,
            MacAddress NVARCHAR(17) NOT NULL,
            Resolution NVARCHAR(20) NOT NULL,
            Orientation NVARCHAR(10) NOT NULL,
            ScreenSizeInch DECIMAL(5,2) NULL,
            Manufacturer NVARCHAR(100) NULL
        );

        -- Parse JSON into temp table
        INSERT INTO #Screen (
            TenantId,
            ScreenName,
            ScreenType,
            PlacementType,
            BasePrice,
            [Status],
            CreatedBy,
            MacAddress,
            Resolution,
            Orientation,
            ScreenSizeInch,
            Manufacturer
        )

        SELECT oj.TenantId,
            oj.ScreenName,
            oj.ScreenType,
            oj.PlacementType,
            oj.BasePrice,
            oj.Status,
            oj.CreatedBy,
            oj.MacAddress,
            oj.Resolution,
            oj.Orientation,
            oj.ScreenSizeInch,
            oj.Manufacturer

        FROM OPENJSON(@Json)
        WITH (
            TenantId INT '$.TenantId',
            ScreenName NVARCHAR(100) '$.ScreenName',

```

```

        ScreenType      NVARCHAR(20)    '$.ScreenType',
        PlacementType   NVARCHAR(20)    '$.PlacementType',
        BasePrice       DECIMAL(10,2)   '$.BasePrice',
        Status          NVARCHAR(20)    '$.Status',
        CreatedBy       INT              '$.CreatedBy',
        MacAddress       NVARCHAR(17)    '$.ScreenInfo.MacAddress',
        Resolution      NVARCHAR(20)    '$.ScreenInfo.Resolution',
        Orientation     NVARCHAR(10)    '$.ScreenInfo.Orientation',
        ScreenSizeInch  DECIMAL(5,2)    '$.ScreenInfo.ScreenSizeInch',
        Manufacturer     NVARCHAR(100)   '$.ScreenInfo.Manufacturer'
    ) AS oj;

-- Reject duplicate MAC address
IF EXISTS (
    SELECT 1
    FROM Inv.ScreenInfo si
    INNER JOIN #Screen ts ON si.MacAddress = ts.MacAddress
)
BEGIN
    ROLLBACK TRANSACTION;
    SET @Json = '{"Type":"Duplicate","Message":"A screen with this MAC address
already exists"}';
    RETURN;
END

-- Insert into Screen and capture inserted Id
INSERT INTO Inv.Screen (
    TenantId,
    ScreenName,
    ScreenType,
    PlacementType,
    BasePrice,
    Status,
    CreatedBy,
    CreatedAt,
    IsDeleted
)
OUTPUT INSERTED.Id INTO #Inserted(Id)
SELECT TenantId, ScreenName, ScreenType, PlacementType, BasePrice, Status,
CreatedBy, GETUTCDATE(), 0
FROM #Screen;

-- Insert into ScreenInfo
INSERT INTO Inv.ScreenInfo (ScreenId, MacAddress, Resolution, Orientation,
ScreenSizeInch, Manufacturer, CreatedAt, IsDeleted)
SELECT i.Id, s.MacAddress, s.Resolution, s.Orientation, s.ScreenSizeInch,
s.Manufacturer, GETUTCDATE(), 0
FROM #Screen s
CROSS JOIN #Inserted i;

-- Return inserted row as JSON
SET @Json = ISNULL(
    (
        SELECT
            s.Id, s.TenantId, t.TenantName,
            s.ScreenName, s.ScreenType, s.PlacementType, s.BasePrice,
s.Status,
            si.MacAddress, si.Resolution, si.Orientation, si.ScreenSizeInch,
si.Manufacturer,
            s.CreatedBy, s.CreatedAt
        FROM   Inv.Screen      s
        JOIN   Inv.ScreenInfo si ON s.Id          = si.ScreenId
    )

```

```

        JOIN Core.Tenant      t  ON s.TenantId = t.Id
        WHERE s.Id = (SELECT Id FROM #Inserted)
        FOR JSON PATH, INCLUDE_NULL_VALUES
    ),
    '[]'
);

COMMIT TRANSACTION;

DROP TABLE IF EXISTS #Inserted;
DROP TABLE IF EXISTS #Screen;

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH
END;

```

3. Bulk Update with Related Table

Create a stored procedure to bulk update records in a main table and also update the corresponding records in the related secondary table.

Include transaction handling to maintain consistency.

```

USE [AdManager_Manish]
GO
/***** Object:  StoredProcedure [Inv].[SpScreenUpd]      Script Date: 3/15/2026 *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
-- =====
-- Author:      Manish Neupane
-- Create date: 14.03.2026
-- Description: Update an existing Screen and its ScreenInfo.
-- Input   : { Id, ScreenName, ScreenType, PlacementType, BasePrice, Status,
--           ScreenInfo: { Resolution, Orientation, ScreenSizeInch, Manufacturer } }
-- Output  : Updated Screen record as JSON array
/*
DECLARE @Json NVARCHAR(MAX) = '{
    "Id":          1,
    "ScreenName":   "Updated Hall Display",
    "ScreenType":   "indoor",
    "PlacementType": "ceiling",
    "BasePrice":    900.00,
    "Status":       "maintenance",
    "ScreenInfo": {
        "Resolution":    "3840x2160",
        "Orientation":    "portrait",
        "ScreenSizeInch": 65.0,
        "Manufacturer":    "LG"
    }
}'
EXEC Inv.SpScreenUpd @Json OUTPUT;
SELECT @Json AS JsonOutput;
*/
-- =====
ALTER PROCEDURE [Inv].[SpScreenUpd]
(

```

```

@Json NVARCHAR(MAX) OUTPUT
)
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;
        SET NOCOUNT ON;

        -- Temp table to capture updated Id
        CREATE TABLE #UpdatedScreen (Id INT);

        -- Temp table to stage parsed JSON data
        CREATE TABLE #Screen (
            Id INT NOT NULL,
            ScreenName NVARCHAR(100) NOT NULL,
            ScreenType NVARCHAR(20) NOT NULL,
            PlacementType NVARCHAR(20) NOT NULL,
            BasePrice DECIMAL(10,2) NOT NULL,
            Status NVARCHAR(20) NOT NULL,
            Resolution NVARCHAR(20) NOT NULL,
            Orientation NVARCHAR(10) NOT NULL,
            ScreenSizeInch DECIMAL(5,2) NULL,
            Manufacturer NVARCHAR(100) NULL
        );

        -- Parse JSON into temp table
        INSERT INTO #Screen (
            Id,
            ScreenName,
            ScreenType,
            PlacementType,
            BasePrice,
            [Status],
            Resolution,
            Orientation,
            ScreenSizeInch,
            Manufacturer
        )

        SELECT oj.Id,
            oj.ScreenName,
            oj.ScreenType,
            oj.PlacementType,
            oj.BasePrice,
            oj.Status,
            oj.Resolution,
            oj.Orientation,
            oj.ScreenSizeInch,
            oj.Manufacturer

        FROM OPENJSON(@Json)
        WITH (
            Id INT '$.Id',
            ScreenName NVARCHAR(100) '$.ScreenName',
            ScreenType NVARCHAR(20) '$.ScreenType',
            PlacementType NVARCHAR(20) '$.PlacementType',
            BasePrice DECIMAL(10,2) '$.BasePrice',
            Status NVARCHAR(20) '$.Status',
            Resolution NVARCHAR(20) '$.ScreenInfo.Resolution',
            Orientation NVARCHAR(10) '$.ScreenInfo.Orientation',
            ScreenSizeInch DECIMAL(5,2) '$.ScreenInfo.ScreenSizeInch',
            Manufacturer NVARCHAR(100) '$.ScreenInfo.Manufacturer'
        ) AS oj;
    
```

```

-- Check if Screen exists and is not deleted
IF NOT EXISTS (
    SELECT 1
    FROM Inv.Screen s
    INNER JOIN #Screen ts ON s.Id = ts.Id
    WHERE s.IsDeleted = 0
)
BEGIN
    ROLLBACK TRANSACTION;
    SET @Json = '{"Type":"NotFound","Message":"Screen not found or has been
deleted"}';
    RETURN;
END

-- Update Screen and capture updated Id
UPDATE s
SET s.ScreenName      = ts.ScreenName,
    s.ScreenType      = ts.ScreenType,
    s.PlacementType   = ts.PlacementType,
    s.BasePrice       = ts.BasePrice,
    s.Status          = ts.Status
OUTPUT INSERTED.Id INTO #UpdatedScreen(Id)
FROM Inv.Screen s
INNER JOIN #Screen ts ON s.Id = ts.Id
WHERE s.IsDeleted = 0;

-- Update ScreenInfo
UPDATE si
SET si.Resolution      = ts.Resolution,
    si.Orientation      = ts.Orientation,
    si.ScreenSizeInch  = ts.ScreenSizeInch,
    si.Manufacturer    = ts.Manufacturer
FROM Inv.ScreenInfo si
INNER JOIN #UpdatedScreen u ON si.ScreenId = u.Id
INNER JOIN #Screen      ts ON ts.Id      = u.Id;

-- Return updated row as JSON
SET @Json = ISNULL(
    (
        SELECT
            s.Id, s.TenantId, t.TenantName,
            s.ScreenName, s.ScreenType, s.PlacementType, s.BasePrice,
s.Status,
            si.MacAddress, si.Resolution, si.Orientation, si.ScreenSizeInch,
si.Manufacturer,
            s.CreatedBy, s.CreatedAt
        FROM   Inv.Screen      s
        JOIN   Inv.ScreenInfo si ON s.Id      = si.ScreenId
        JOIN   Core.Tenant    t  ON s.TenantId = t.Id
        WHERE  s.Id = (SELECT Id FROM #UpdatedScreen)
        FOR JSON PATH, INCLUDE_NULL_VALUES
    ),
    '[]'
);

COMMIT TRANSACTION;

DROP TABLE IF EXISTS #UpdatedScreen;
DROP TABLE IF EXISTS #Screen;

END TRY

```



```

        BEGIN CATCH
            IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
            THROW;
        END CATCH
    END;

```

4. Bulk Delete with Related Records

Create a stored procedure to bulk delete records from the main table and also delete the associated records in the related table.

```

USE [AdManager_Manish]
GO
/***** Object:  StoredProcedure [Inv].[SpScreenDel]      Script Date: 3/15/2026 *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
-- =====
-- Author:      Manish Neupane
-- Create date: 14.03.2026
-- Description: Soft-delete a Screen by setting IsDeleted = 1.
-- Input   : { Id, UpdatedBy }
-- Output  : Deleted Screen record as JSON array
/*
DECLARE @Json NVARCHAR(MAX) = '{
    "Id":      1,
    "UpdatedBy": 1
}'
EXEC Inv.SpScreenDel @Json OUTPUT;
SELECT @Json AS JsonOutput;
*/
-- =====
ALTER PROCEDURE [Inv].[SpScreenDel]
(
    @Json NVARCHAR(MAX) OUTPUT
)
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;
        SET NOCOUNT ON;

        -- Temp table to capture deleted Id
        CREATE TABLE #DeletedScreen (Id INT);

        -- Temp table to stage parsed JSON data
        CREATE TABLE #Screen (
            Id      INT NOT NULL,
            UpdatedBy INT NOT NULL
        );

        -- Parse JSON into temp table
        INSERT INTO #Screen (
            Id,
            UpdatedBy
        )
        SELECT  oj.Id,
                oj.UpdatedBy
    
```

```

FROM OPENJSON(@Json)
WITH (
    Id INT '$.Id',
    UpdatedBy INT '$.UpdatedBy'
) AS oj;

-- Check if Screen exists and is not deleted
IF NOT EXISTS (
    SELECT 1
    FROM Inv.Screen s
    INNER JOIN #Screen ts ON s.Id = ts.Id
    WHERE s.IsDeleted = 0
)
BEGIN
    ROLLBACK TRANSACTION;
    SET @Json = '{"Type":"NotFound","Message":"Screen not found or already
deleted"}';
    RETURN;
END

-- Soft-delete Screen and capture deleted Id
UPDATE s
SET s.IsDeleted = 1
OUTPUT INSERTED.Id INTO #DeletedScreen(Id)
FROM Inv.Screen s
INNER JOIN #Screen ts ON s.Id = ts.Id
WHERE s.IsDeleted = 0;

-- Return deleted row as JSON
SET @Json = ISNULL(
    (
        SELECT
            s.Id, s.TenantId, t.TenantName,
            s.ScreenName, s.Status, s.IsDeleted,
            s.CreatedBy, s.CreatedAt
        FROM Inv.Screen s
        JOIN Core.Tenant t ON s.TenantId = t.Id
        WHERE s.Id = (SELECT Id FROM #DeletedScreen)
        FOR JSON PATH, INCLUDE_NULL_VALUES
    ),
    '[]'
);

COMMIT TRANSACTION;

DROP TABLE IF EXISTS #DeletedScreen;
DROP TABLE IF EXISTS #Screen;

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH
END;

```

5. Dropdown Data Retrieval

Create a stored procedure to fetch data for a dropdown list, returning only the necessary fields (e.g., Id and Name).

```

USE [AdManager_Manish]
GO
/***** Object:  StoredProcedure [Inv].[SpScreenDropSel]      Script Date: 3/15/2026
6:05:53 PM *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
-- =====
-- Author:      Manish Neupane
-- Create date: 14.03.2026
-- Description: Active Screens as Id + Name list for dropdowns.
-- Input   : {} (empty Json -- ignored)
-- Output  : JSON array of { Id, Name }
/*
DECLARE @Json NVARCHAR(MAX) = N'{}'
EXEC Inv.SpScreenDropSel @Json;
*/
-- =====
ALTER PROCEDURE [Inv].[SpScreenDropSel]
(
    @Json NVARCHAR(MAX) = NULL
)
AS
BEGIN
    BEGIN TRY
        SET NOCOUNT ON;

        DECLARE @Result NVARCHAR(MAX);

        SET @Result = ISNULL(
            (
                SELECT
                    s.Id,
                    s.ScreenName AS Name
                FROM   Inv.Screen s
                WHERE  s.IsDeleted = 0
                    AND s.Status   = 'active'
                ORDER BY s.ScreenName ASC
                FOR JSON PATH
            ),
            '[]'
        );

        SELECT @Result;

    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;

```

6. Upsert (Insert or Update) Operation

Create a stored procedure to insert a new record if it does not exist, or update the existing record if a duplicate key is found.

Use transactions and proper error handling.

```

USE [AdManager_Manish]

```

```

GO
/***** Object:  StoredProcedure [Inv].[SpScreenTsk]      Script Date: 3/15/2026 *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
-- =====
-- Author:      Manish Neupane
-- Create date: 14.03.2026
-- Description: Upsert a Screen. Inserts if Id is null, updates if Id is provided.
-- Input  : { Id (null = insert), TenantId, ScreenName, ScreenType, PlacementType,
--           BasePrice, Status, CreatedBy,
--           ScreenInfo: { MacAddress, Resolution, Orientation, ScreenSizeInch,
Manufacturer } }
-- Output : Affected Screen record as JSON array
/*
DECLARE @Json NVARCHAR(MAX) = '{
    "TenantId":      1,
    "ScreenName":    "Rooftop Board",
    "ScreenType":    "outdoor",
    "PlacementType": "freestanding",
    "BasePrice":     1200.00,
    "Status":        "active",
    "CreatedBy":     1,
    "ScreenInfo": {
        "MacAddress": "FF:EE:DD:CC:BB:01",
        "Resolution": "3840x2160",
        "Orientation": "landscape",
        "ScreenSizeInch": 75.0,
        "Manufacturer": "Samsung"
    }
}'
EXEC Inv.SpScreenTsk @Json OUTPUT;
SELECT @Json AS JsonOutput;
*/
-- =====
ALTER PROCEDURE [Inv].[SpScreenTsk]
(
    @Json NVARCHAR(MAX) OUTPUT
)
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;
        SET NOCOUNT ON;

        -- Temp table to capture affected Id
        CREATE TABLE #AffectedScreen (Id INT);

        -- Temp table to stage parsed JSON data
        CREATE TABLE #Screen (
            Id INT NULL,
            TenantId INT NOT NULL,
            ScreenName NVARCHAR(100) NOT NULL,
            ScreenType NVARCHAR(20) NOT NULL,
            PlacementType NVARCHAR(20) NOT NULL,
            BasePrice DECIMAL(10,2) NOT NULL,
            Status NVARCHAR(20) NOT NULL,
            CreatedBy INT NOT NULL,
            MacAddress NVARCHAR(17) NOT NULL,
            Resolution NVARCHAR(20) NOT NULL,
            Orientation NVARCHAR(10) NOT NULL,
            ScreenSizeInch DECIMAL(5,2) NULL,

```

```

        Manufacturer    NVARCHAR(100)    NULL
    );

-- Parse JSON into temp table
INSERT INTO #Screen (
    Id,
    TenantId,
    ScreenName,
    ScreenType,
    PlacementType,
    BasePrice,
    [Status],
    CreatedBy,
    MacAddress,
    Resolution,
    Orientation,
    ScreenSizeInch,
    Manufacturer
)

SELECT  oj.Id,
        oj.TenantId,
        oj.ScreenName,
        oj.ScreenType,
        oj.PlacementType,
        oj.BasePrice,
        oj.Status,
        oj.CreatedBy,
        oj.MacAddress,
        oj.Resolution,
        oj.Orientation,
        oj.ScreenSizeInch,
        oj.Manufacturer

FROM OPENJSON (@Json)
WITH (
    Id                INT                '$.Id',
    TenantId          INT                '$.TenantId',
    ScreenName         NVARCHAR(100)     '$.ScreenName',
    ScreenType         NVARCHAR(20)      '$.ScreenType',
    PlacementType      NVARCHAR(20)      '$.PlacementType',
    BasePrice          DECIMAL(10,2)     '$.BasePrice',
    Status             NVARCHAR(20)      '$.Status',
    CreatedBy          INT                '$.CreatedBy',
    MacAddress         NVARCHAR(17)      '$.ScreenInfo.MacAddress',
    Resolution         NVARCHAR(20)      '$.ScreenInfo.Resolution',
    Orientation        NVARCHAR(10)      '$.ScreenInfo.Orientation',
    ScreenSizeInch     DECIMAL(5,2)      '$.ScreenInfo.ScreenSizeInch',
    Manufacturer       NVARCHAR(100)     '$.ScreenInfo.Manufacturer'
) AS oj;

-- INSERT path
IF (SELECT Id FROM #Screen) IS NULL
BEGIN
    -- Check for duplicate MAC address
    IF EXISTS (
        SELECT 1
        FROM Inv.ScreenInfo si
        INNER JOIN #Screen ts ON si.MacAddress = ts.MacAddress
    )
    BEGIN
        ROLLBACK TRANSACTION;
        SET @Json = '{"Type":"Duplicate","Message":"A screen with this MAC
address already exists"}';
    
```

```

        RETURN;
    END

    -- Insert Screen and capture inserted Id
    INSERT INTO Inv.Screen (
        TenantId,
        ScreenName,
        ScreenType,
        PlacementType,
        BasePrice,
        [Status],
        CreatedBy,
        CreatedAt,
        IsDeleted
    )
    OUTPUT INSERTED.Id INTO #AffectedScreen(Id)
    SELECT  ts.TenantId,
            ts.ScreenName,
            ts.ScreenType,
            ts.PlacementType,
            ts.BasePrice,
            ts.Status,
            ts.CreatedBy,
            GETUTCDATE(),
            0
    FROM #Screen ts;

    -- Insert ScreenInfo
    INSERT INTO Inv.ScreenInfo (
        ScreenId,
        MacAddress,
        Resolution,
        Orientation,
        ScreenSizeInch,
        Manufacturer,
        CreatedAt,
        IsDeleted
    )
    SELECT  a.Id,
            ts.MacAddress,
            ts.Resolution,
            ts.Orientation,
            ts.ScreenSizeInch,
            ts.Manufacturer,
            GETUTCDATE(),
            0
    FROM #Screen ts
    CROSS JOIN #AffectedScreen a;
END
ELSE
BEGIN
    -- Check if Screen exists and is not deleted
    IF NOT EXISTS (
        SELECT 1
        FROM Inv.Screen s
        INNER JOIN #Screen ts ON s.Id = ts.Id
        WHERE s.IsDeleted = 0
    )
    BEGIN
        ROLLBACK TRANSACTION;
        SET @Json = '{"Type":"NotFound","Message":"Screen not found or has
been deleted"}';
        RETURN;
    END

```

```

END

-- Update Screen and capture updated Id
UPDATE s
SET s.ScreenName      = ts.ScreenName,
    s.ScreenType      = ts.ScreenType,
    s.PlacementType   = ts.PlacementType,
    s.BasePrice       = ts.BasePrice,
    s.Status          = ts.Status
OUTPUT INSERTED.Id INTO #AffectedScreen(Id)
FROM Inv.Screen s
INNER JOIN #Screen ts ON s.Id = ts.Id
WHERE s.IsDeleted = 0;

-- Update ScreenInfo
UPDATE si
SET si.Resolution     = ts.Resolution,
    si.Orientation    = ts.Orientation,
    si.ScreenSizeInch = ts.ScreenSizeInch,
    si.Manufacturer   = ts.Manufacturer
FROM Inv.ScreenInfo si
INNER JOIN #AffectedScreen a ON si.ScreenId = a.Id
INNER JOIN #Screen        ts ON ts.Id      = a.Id;
END

-- Return affected row as JSON
SET @Json = ISNULL(
    (
        SELECT
            s.Id, s.TenantId, t.TenantName,
            s.ScreenName, s.ScreenType, s.PlacementType, s.BasePrice,
s.Status,
            si.MacAddress, si.Resolution, si.Orientation, si.ScreenSizeInch,
si.Manufacturer,
            s.CreatedBy, s.CreatedAt
        FROM   Inv.Screen      s
        JOIN   Inv.ScreenInfo si ON s.Id      = si.ScreenId
        JOIN   Core.Tenant    t  ON s.TenantId = t.Id
        WHERE  s.Id = (SELECT Id FROM #AffectedScreen)
        FOR JSON PATH, INCLUDE_NULL_VALUES
    ),
    '[]'
);

COMMIT TRANSACTION;

DROP TABLE IF EXISTS #AffectedScreen;
DROP TABLE IF EXISTS #Screen;

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH
END;

```

7. Search with Optional Filters

Create a stored procedure that retrieves data from a table with optional filter parameters (e.g., name, status, date range).

Only apply filters when values are provided.

```
USE [AdManager_Manish]
GO
/***** Object:  StoredProcedure [Inv].[SpScreenFilterSel]      Script Date: 3/15/2026
6:00:55 PM *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
-- =====
-- Author:      Manish Neupane
-- Create date: 13.03.2026
-- Description: Search Screens with optional scalar filters and free-text search.
-- Input   : { ScreenType, Status, TenantId, SearchText }
-- Output  : JSON array of matching Screens
/*
DECLARE @Json NVARCHAR(MAX) = N'{
    "ScreenType": "indoor",
    "Status":     "active",
    "TenantId":   null,
    "SearchText": "lobby"
}'
EXEC Inv.SpScreenFilterSel @Json;
*/
-- =====
ALTER PROCEDURE [Inv].[SpScreenFilterSel]
(
    @Json NVARCHAR(MAX)
)
AS
BEGIN
    BEGIN TRY
        SET NOCOUNT ON;

        DECLARE
            @ScreenType NVARCHAR(20) = JSON_VALUE(@Json, '$.ScreenType'),
            @Status      NVARCHAR(20) = JSON_VALUE(@Json, '$.Status'),
            @TenantId    INT          = TRY_CAST(JSON_VALUE(@Json, '$.TenantId') AS
INT),
            @SearchText  NVARCHAR(100) = JSON_VALUE(@Json, '$.SearchText'),
            @Result       NVARCHAR(MAX);

        SET @Result = ISNULL(
            (
                SELECT
                    s.Id,
                    s.TenantId,
                    t.TenantName,
                    s.ScreenName,
                    s.ScreenType,
                    s.PlacementType,
                    s.BasePrice,
                    s.Status,
                    si.MacAddress,
                    si.Resolution,
                    si.Orientation,
                    si.ScreenSizeInch,
                    si.Manufacturer,
                    s.CreatedAt,
                    s.CreatedBy
                FROM Inv.Screen      s
```



```

        JOIN Inv.ScreenInfo si ON s.Id          = si.ScreenId
        JOIN Core.Tenant      t  ON s.TenantId = t.Id
        WHERE s.IsDeleted = 0
              AND (@ScreenType IS NULL OR s.ScreenType = @ScreenType)
              AND (@Status      IS NULL OR s.Status    = @Status)
              AND (@TenantId    IS NULL OR s.TenantId  = @TenantId)
              AND (@SearchText IS NULL
                  OR s.ScreenName LIKE '%' + @SearchText + '%'
                  OR si.Manufacturer LIKE '%' + @SearchText + '%'
                  OR si.MacAddress LIKE '%' + @SearchText + '%')
        ORDER BY s.CreatedAt DESC
        FOR JSON PATH, INCLUDE_NULL_VALUES
    ),
    '[]'
);

SELECT @Result;

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH
END;

```